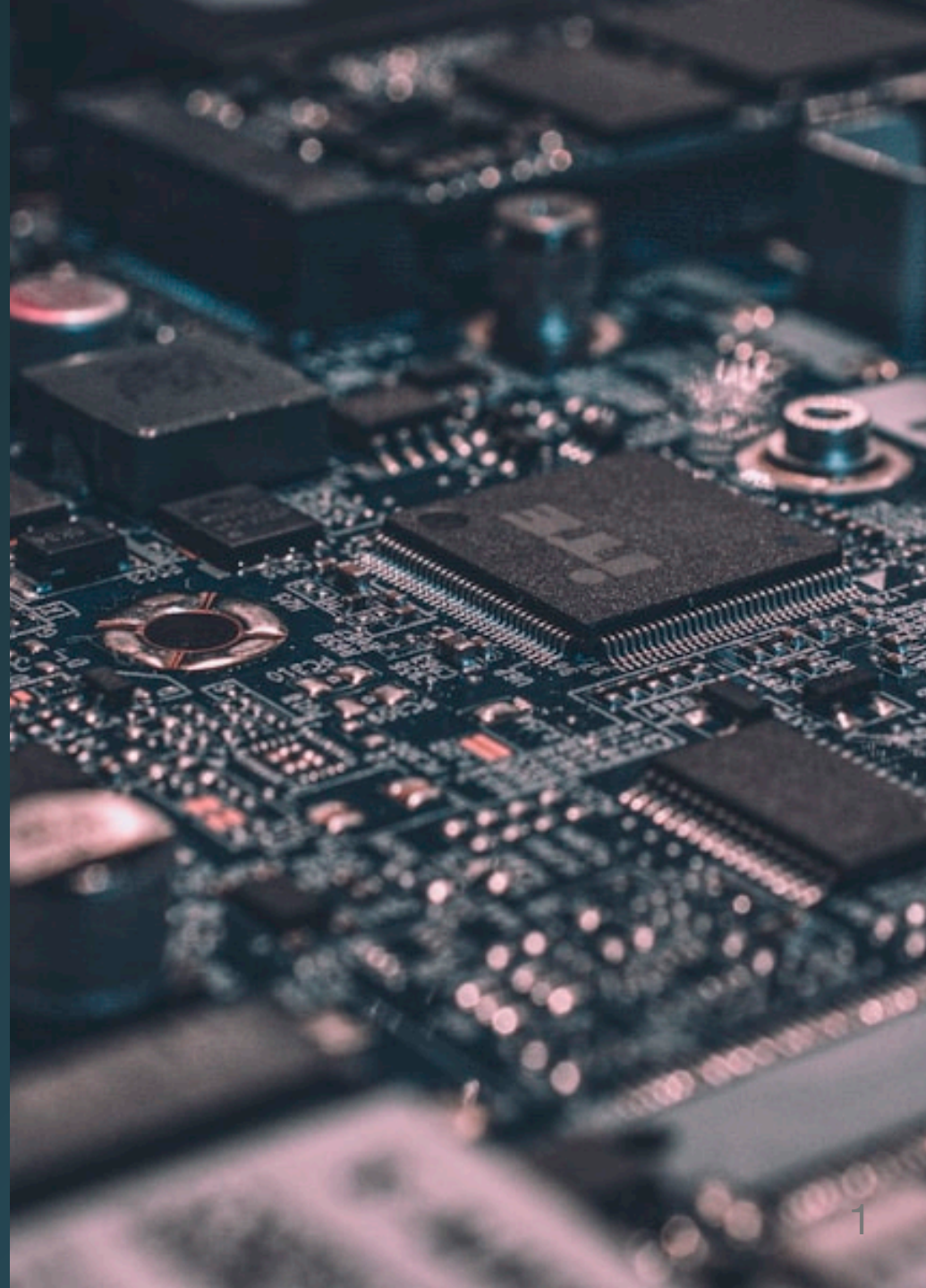
Scrape Pipeline

Lesson 09 · ISBA 4715



Agenda

	Part	What
1	**Scraping Concepts**	What it is, etiquette, the tools we'll use *(slides)*
2	**Python Pipeline**	Firecrawl search + scrape, saving to `knowledge/raw/` *(live code)*
3	**MCP Upgrade**	Claude Code as the pipeline *(demo)*
4	**Your Project**	Scrape at least one source into your repo

How did Google build its first index?

Before anyone wrote an API for the web...

How did Google build its first index?

Before anyone wrote an API for the web...

Web scraping — reading pages programmatically.

Web Scraping

> Gathering data from websites **without an API** and **without a human clicking around in a browser**

Why Scrape?

No API exists for this data

Constantly-changing list (press releases, bios, news)

The API is paywalled

You want to automate something a human would do

Etiquette and Caveats

Rule	Why
Check `robots.txt`	Sites declare what bots can crawl
Rate limit — `time.sleep()`	Don't hammer the server
Respect Terms of Service	Legal + ethical obligation
Expect layout changes	HTML is not a stable contract
Be mindful of PII	Don't scrape what people did not consent to share

Where does scraped data go in THIS project?

Where does scraped data go?

MP03 (API)

request → CSV → SQL warehouse

Where does scraped data go?

MP03 (API)

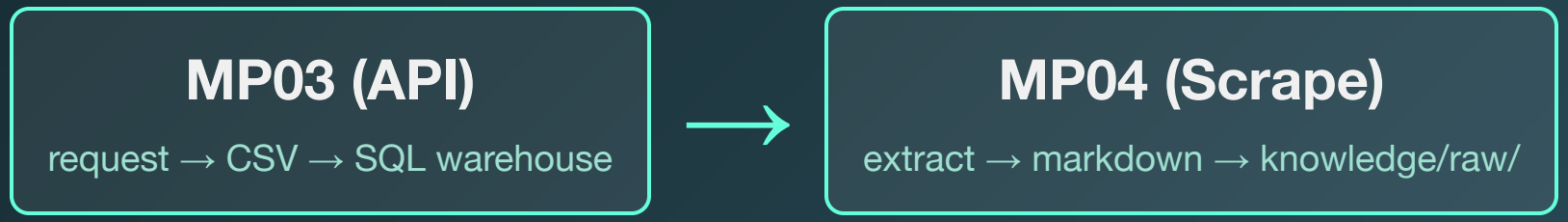
request → CSV → SQL warehouse

→

MP04 (Scrape)

extract → markdown → knowledge/raw/

Where does scraped data go?



These feed your **knowledge base** — the unstructured side of your portfolio project.

You are building a RAG system

Retrieval-**A**ugmented **G**eneration — the standard pattern for AI systems that answer questions using your own content rather than just what a model memorized during training.

Every knowledge-base chatbot, internal-docs search, and "chat with your data" product you have seen is RAG.

How we got here

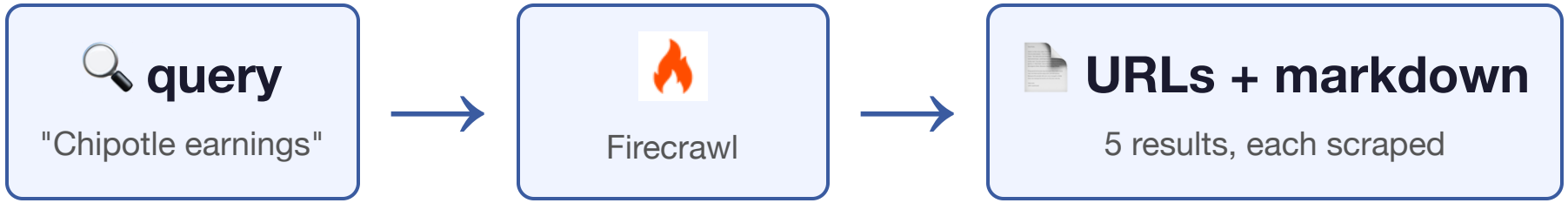
LLMs on their own confabulate. RAG grounds answers in documents you supply. Your project is the agentic flavor — Claude Code chooses which files in `knowledge/raw/` to read for each question.

Your MP04 pipeline, in RAG terms

- **Retrieval** — Firecrawl searches the web and pulls back markdown
- **Document store** — `knowledge/raw/` holds the scraped markdown
- **Indexed notes** — `knowledge/wiki/` (you build this in Milestone 02)
- **Generation** — Claude Code reads the notes and answers grounded in them

No vector database, no embeddings — Claude Code uses grep, file reads, and its own reasoning. Simple RAG beats complex RAG until you have a reason otherwise.

One API, Search + Extract



Older tools split this into two calls — one service for search, another for scraping. Firecrawl's `search` endpoint does both in one round-trip.

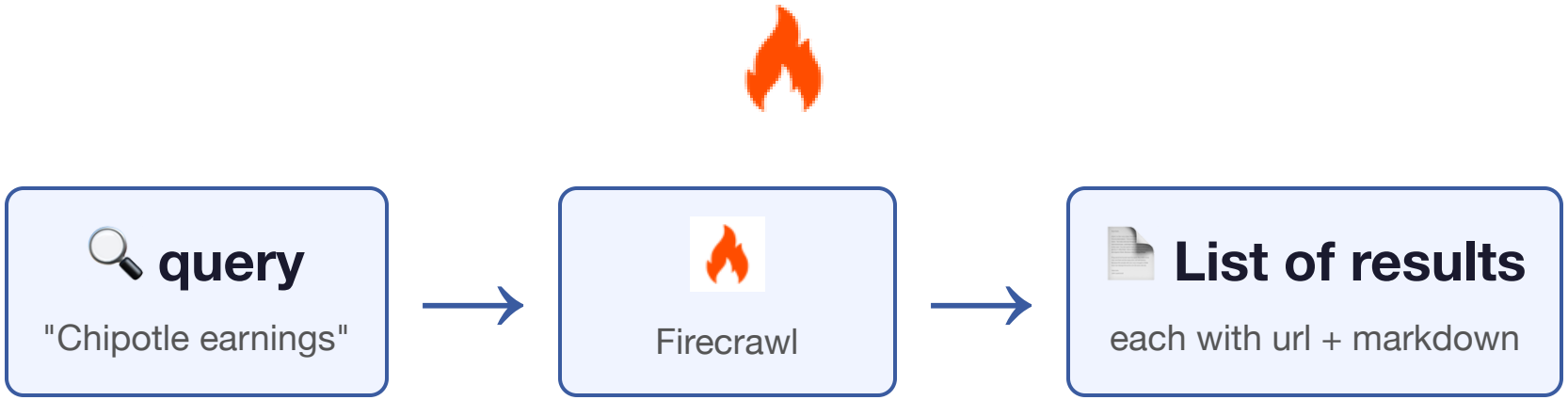
Firecrawl vs. Claude Code's WebSearch

 vs. 

Firecrawl (API)	WebSearch (Claude Code tool)
Returns structured data (URLs, titles, markdown)	Returns prose for the agent to read
Designed for programmatic pipelines	Built into Claude Code conversations
Same response schema every call	Different synthesized text each call
Full control: filters, domains, depth	Whatever the tool decides

> Cron jobs need structured data to parse. Humans need readable prose. Pick the right tool for the reader.

Firecrawl — Search + Scrape API



No BeautifulSoup. No `select` or `find`. No DOM inspection.

Last Year We Taught BeautifulSoup

- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns

Last Year We Taught BeautifulSoup

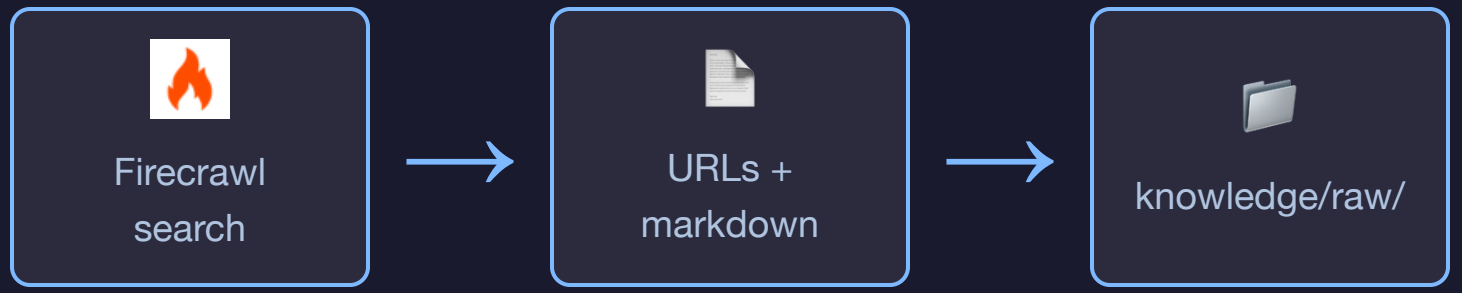
- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns

This year we don't. Firecrawl does all three in one call, and handles JavaScript rendering too. If you want BeautifulSoup for interview prep, read the docs — you do not need it for this project.

Build vs. Buy — Scraping Edition

Problem	Don't	Do
🔍 Find + scrape URLs	Write a crawler + parser	**Firecrawl**
🖱️ Click through a login	Write a bot	Playwright

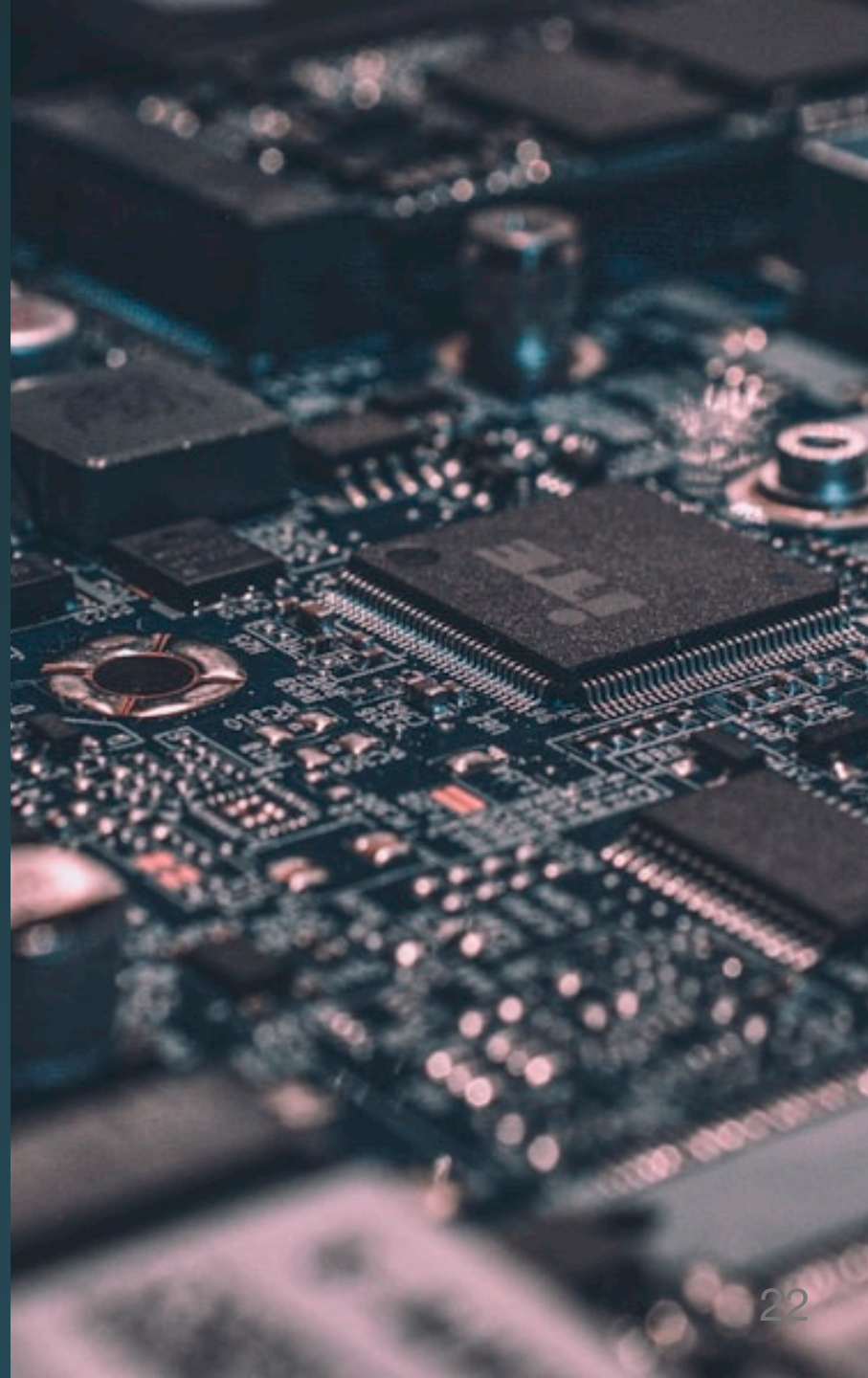
The Pipeline



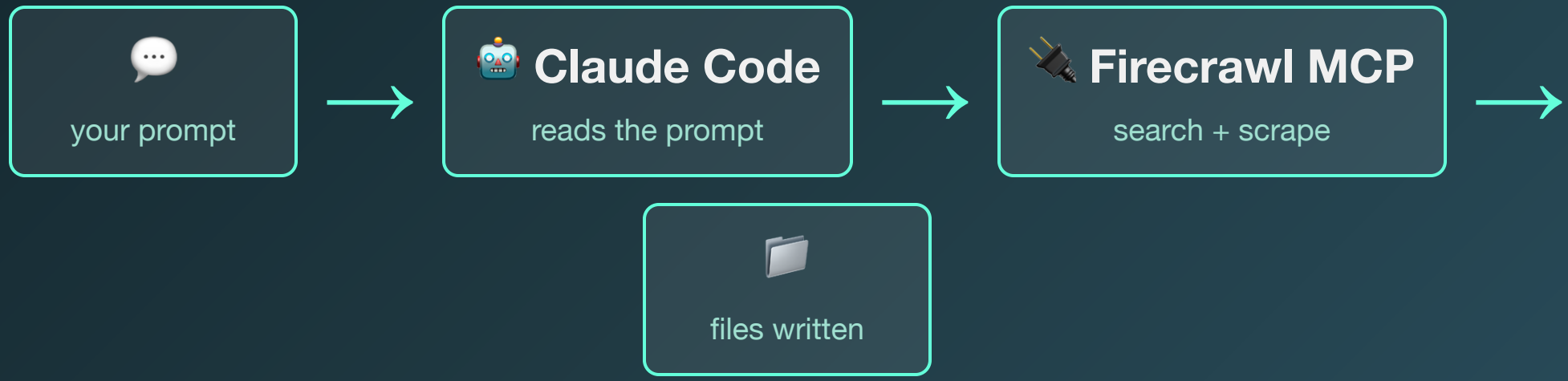
One API does the finding and the extracting. Your script saves the output.

Now the Upgrade: MCP

What if Claude Code could call Firecrawl **directly**, without you writing Python?



Now the Upgrade: MCP



MCP servers (Model Context Protocol) extend Claude Code with new tools. Install one, and Claude Code can use it inside any prompt.

Python vs. MCP

🐍 Python (Part 02)

- ~30 lines of SDK calls
- Loops, file writes
- Version-controlled
- Runs in GitHub Actions

💬 MCP (Part 03)

> "Find 5 URLs about Chipotle's recent earnings and save each as markdown in `knowledge/raw/`."

Same result. No code.

Today's Demo Domain: Chipotle IR

- **IR = Investor Relations** — shareholder-facing content
- Press releases, bios, earnings, filings
- Legally required to be accessible
- Great fit for a knowledge base

Accounts You Need

Service	URL	Free Tier
Firecrawl	firecrawl.dev	500 credits
Student Program	firecrawl.dev/student-program	**20,000 credits** with `.edu` email

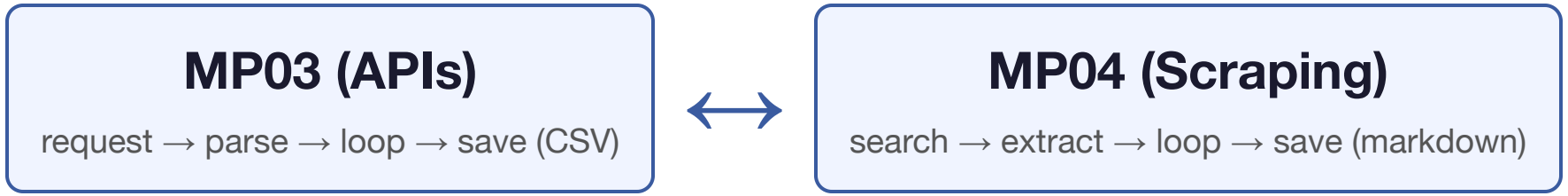
To redeem student credits: Dashboard → **Settings** → **Billing** → apply coupon code `STUDENTEDU`.

Sign up with your `.edu` email (not GitHub OAuth) so the coupon can verify your school. No card required.

Your Project Needs This

- 🎯 Milestone 02: **≥15 sources** from 3+ sites
- 🔗 Pattern works for any URL in a browser
- 💼 Your job posting says what content matters
- 📚 Feeds your knowledge base wiki

The Pattern Transfers



Different tools. Same shape. If you learned MP03, you already know MP04.

Next up: code it together.